A Queueing-Network Simulator for an LLM Inference Serving Pipeline: Phase-2 Progress Report

Vaibhav Mahore (23631)*, Sujal Chavda (24288)*, Arindam Raina (25880)*, Shreemaye Das (25958)†
*Department of Computer Science and Automation (CSA), Indian Institute of Science, Bangalore, India
†Department of Computational and Data Sciences (CDS), Indian Institute of Science, Bangalore, India
Course: E0 240 (Modeling and Simulation) Email: {vaibhavm, sujals, arindamr, shreemayed}@iisc.ac.in

***Abstract*—Large language model (LLM) inference serving systems must satisfy strict Time-to-First-Token (TTFT) and Inter-Token Latency (ITL) Service Level Objectives (SLOs) under bursty arrivals and variable generation lengths. We design and implement a cycle-driven discrete-time queueing-network simulator that models the serving pipeline as an open three-station tandem network: a deterministic admission and tokenization server (Q_1: $Geo/D/1$, S_1=2), a dual-trigger First-Come First-Served (FCFS) batching scheduler (Q_2: cap B, timeout τ), and a parallel multi-worker GPU decode pool (Q_3: $Geo/Geo/c$ or sublinear batch service $S_3(b)$). Directly addressing Phase-1 feedback, we establish a structural systems-to-queueing mapping that decomposes TTFT and ITL into station-level waiting delays ($W_{q,1}, W_{q,2}, W_{q,3}$) and service times. We then execute a two-layer validation and experimental study. In Layer 1, we verify our engine against discrete-time closed forms over 10^7 cycles, achieving worst-case occupancy errors of 0.75% ($Geo/D/1$, S_1=2), 0.46% (S_1=3), and 0.94% ($Geo/Geo/1$, $E[S]$=2), reproducing the Pollaczek-Khinchine factor-2 variance penalty within 0.31% at λ=0.45 and network flow conservation within 3.4×10^{-7}. In Layer 2, we ground service scaling in batched inference sweeps across $B \in \{1, 2, 4, 8, 16, 32\}$ on an Nvidia Tesla T4 GPU across four model families (`Qwen2.5-1.5B`, `Gemma-2-2B`, `Llama-3.2-1B`, and `Phi-3-mini-4k`). Calibrating a power-law model on `Qwen2.5-1.5B` ($B \in \{1, 4, 16\}$) predicts held-out sizes $\{2, 8, 32\}$ with 4.43% latency MAPE and 4.18% throughput MAPE, while an uncapped 100-prompt diagnostic (512-token ceiling) reveals non-geometric output lengths (KS=0.3965) where straggler length $\max_{j\le B} L_j$ governs batch runtime (r=0.9493, cutting uncapped latency MAPE from 15.25% to 9.14%). Finally, experiments E2 and E3 prove that c=2 parallel workers absorb an 80.2% service-variability penalty at λ=0.45, while static batching (τ=8, $B\ge$8) cuts GPU worker utilization by 48.9% ($0.5999 \to 0.3063$).**

I. Problem Statement and Motivation

Serving generative Large Language Models (LLMs) such as LLaMA and Qwen in interactive applications requires satisfying strict latency Service Level Objectives (SLOs) under stochastic request arrivals [1], [4]: *Time-to-First-Token* (TTFT), measuring initial responsiveness, and *Inter-Token Latency* (ITL), measuring token generation speed.

A. Why Model LLM Serving as a Queueing Network? (Addressing Phase-1 Feedback #1)

A central question raised in our Phase-1 evaluation is: *why should an LLM inference pipeline be modeled as a network of queues rather than analyzed as a single compute kernel or evaluated solely through full-system profiling?* We establish three foundational justifications, illustrated in Fig. 1:

1) Physical Decoupling of Asynchronous Resource Stages. In production serving stacks (vLLM [1], Hugging Face TGI, TensorRT-LLM), requests traverse three decoupled stages governed by distinct queueing mechanics:

- *CPU Admission & Tokenization (Q_1):* An asynchronous front-end parses HTTP/gRPC payloads and executes Byte-Pair Encoding (BPE) tokenization on the CPU. Because tokenization takes a small, deterministic slice (0.206 ms mean in our T4 host trials) decoupled from GPU contention, it forms a single-server deterministic queue ($Geo/D/1$).
- *Scheduler Batch Accumulation (Q_2):* Autoregressive decoding is memory-bandwidth bound on GPU High-Bandwidth Memory (HBM); serving B=1 wastes compute capacity. The scheduler therefore buffers tokenized requests in an FCFS queue until B requests accumulate or a latency timer τ expires [2], introducing an explicit *batch-aggregation delay* $W_{q,2}$.
- *Parallel GPU Execution Contention (Q_3):* Formed batches are dispatched to c parallel GPU execution streams. Because output token lengths vary widely across prompts (83–487 tokens in our uncapped diagnostic), active batches occupy workers for stochastic durations, subjecting arriving batches to *worker contention delay* $W_{q,3}$.

2) Exact Additive Decomposition of User-Perceived Latency. End-to-end request sojourn time W_{soj} decomposes additively across all three stations:

$$W_{\text{soj}} = \underbrace{W_{q,1} + W_{q,2} + W_{q,3}}_{W_{\text{e2e,qwait}}\ \text{(Queueing Delay)}} + \underbrace{S_1 + S_3(b, L_{\max})}_{\text{Service Duration}}. \quad (1)$$

As load approaches capacity ($\rho_i \to 1$), queueing delay $W_{\text{e2e,qwait}}$ dominates hardware compute time by over $15\times$ (growing from 0.057 cycles at λ=0.05 to 65.15 cycles at λ=0.49 in Experiment E2). A multi-station network isolates whether tail latency stems from admission bottlenecks (Q_1), overly patient batching timers (τ at Q_2), or decode-length variance amplification (Q_3).

3) Analytical Tractability and Speed vs. Operator Profiling. Trace-driven simulators like Vidur [4] profile thousands of tensor operators across GPU clusters, requiring costly hardware access and offering no closed-form insight into stability limits. Our discrete-time queueing simulator evaluates 10^7 cycles in ≈ 6.5 s on a single CPU core, sweeping (λ, B, τ, c) analytically and numerically while grounding service laws in real Tesla T4 GPU measurements.

II. Related Work

LLM Serving Systems. Orca [2] pioneered iteration-level (continuous) scheduling to retire completed sequences at token boundaries. vLLM [1] introduced PagedAttention to eliminate KV-cache fragmentation, unlocking larger batch caps B. FlexGen [3] maximized single-GPU offline throughput via CPU/disk offloading. While these systems engineer runtime

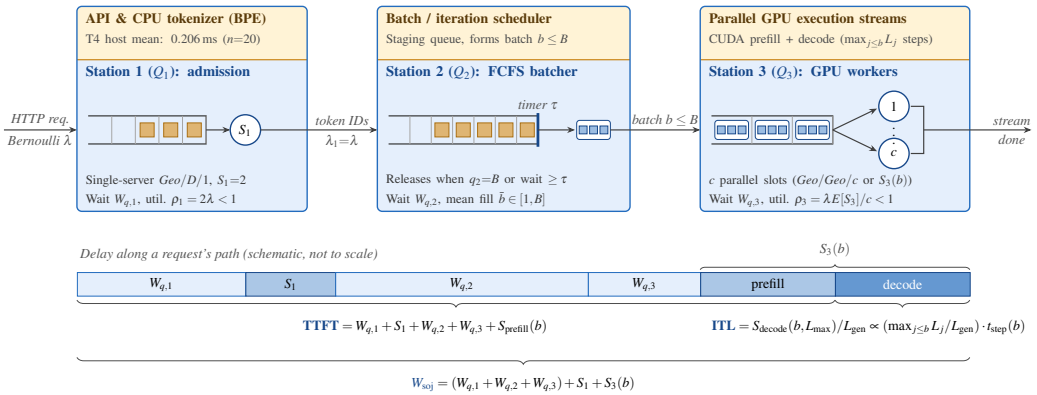


Fig. 1. Structural mapping from a physical LLM serving stack (amber strips) to our three-station discrete-time queueing network (blue bodies), addressing Phase-1 Feedback #1. Squares are requests, rounded boxes are batches and circles are servers. The bar splits a request's delay into queueing waits $W_{q,i}$ and service times (not to scale) and shows how TTFT, ITL and end-to-end sojourn (W_{soj}) are composed.

mechanisms, our work formulates a verifiable queueing model of their batching and worker-pool trade-offs.

Simulators and Queueing Theory. Vidur [4] predicts cluster scheduling latency via machine-learned operator tables but lacks closed-form queueing verification. Following E0 240's discrete-time modeling principles [5], we verify our baseline nodes against exact $Geo/D/1$ and $Geo/Geo/1$ closed forms and calibrate Q_3 using empirical Tesla T4 traces [6].

III. Methodology and Simulator Architecture

A. Three-Station Tandem Queueing Formulation

Time advances in discrete cycles $t \in \{1, \dots, T\}$. External arrivals follow a Bernoulli process with rate $\lambda \in (0, 0.5)$ requests/cycle. **Station 1 (Q_1):** Single-server FIFO $Geo/D/1$ queue with deterministic service $S_1 = 2$ cycles ($\mu_1 = 0.5$; $S_1 = 3$ also verified), using the course's same-cycle immediate-access convention (`single_queue_simulation_geo_d_1.m`). **Station 2 (Q_2):** FCFS batch buffer holding $q_2(t)$ requests with head-of-line arrival cycle t_{head}. At cycle t, Q_2 releases a batch of size b to Q_3 when either trigger fires:

$$q_2(t) \ge B \implies b = B, \quad \text{or} \quad t - t_{\text{head}} \ge \tau \implies b = q_2(t) < B. \tag{2}$$

With B=1 or τ=0, Q_2 is a zero-delay pass-through ($W_{q,2}$=0). **Station 3 (Q_3):** Pool of c homogeneous parallel workers, each serving one entire batch synchronously (all requests in the batch share the batch service duration) under either: (i) a *deterministic sublinear law* $S_3(b) = \lceil S_{\text{base}} + \delta(b-1) \rceil$ (S_{base}=2), or (ii) a *geometric law* with completion probability $p_s(b) = 1/(S_{\text{base}} + \delta(b-1))$ and mean $E[S_3(b)] = 1/p_s(b)$.

Stability Criteria. Because requests are conserved along the path ($\lambda_1 = \lambda_2 = \lambda_3 = \lambda$), the binding stability conditions are:

$$\rho_1 = \lambda S_1 = 2\lambda < 1 \implies \lambda < 0.50, \qquad \rho_3 = \frac{\lambda E[S_3]}{c} < 1, \tag{3}$$

where batch service duration $S_3(b)$ depends on the realized batch size b, and Q_2 is stable whenever Q_3 is stable.

B. Cycle-Driven Engine and Common Random Numbers (CRN)

In each cycle t, `network_sim.py` executes four steps in exact class-code order: (1) decrement deterministic countdowns or draw geometric completions via Inverse Transform sampling ($U \le p_s$); (2) route completed jobs downstream ($Q_1 \to Q_2 \to Q_3 \to$ Exit); (3) check Q_2 release triggers and dispatch queued jobs to idle workers; and (4) accumulate post-dispatch queue lengths $Q_i(t)$ (excluding in-service requests) and busy worker counts $B_i(t)$. Time-average occupancy $L_{q,i} = \frac{1}{T}\sum_{t=1}^{T} Q_i(t)$, mean queue wait $W_{q,i} = \frac{1}{N_i}\sum_{k=1}^{N_i} D_{i,k}$, and utilization $\hat{\rho}_i = \frac{1}{T c_i}\sum_{t=1}^{T} B_i(t)$ use fixed seed `20260922`. Multi-config sweeps (E2, E3) use *Common Random Numbers (CRN)* so all compared configurations share identical arrival streams.

TABLE I
Layer-1 Analytical & Network Accounting Verification (10^7 Cycles, Seed `20260922`)

Validation Check & Configuration	Metric / Relation	Analytical	Simulated	Rel. Error
V1: Single-Station Deterministic Service ($Geo/D/1$)				
S_1=2, λ=0.30 (ρ=0.60)	$L_q = \lambda^2/(1-2\lambda)$	0.2250	0.2260	0.46%
S_1=2, λ=0.30 (ρ=0.60)	$W_q = \rho(S-1)/(2(1-\rho))$	0.7500	0.7530	0.41%
S_1=2, λ=0.45 (ρ=0.90)	$L_q = \lambda^2/(1-2\lambda)$	2.0250	2.0324	0.37%
S_1=2, worst stable (λ=0.35)	$\max_{\lambda \le 0.45} \lvert L_q - \hat{L}_q \rvert / L_q$	0.4083	0.4053	0.75%
S_1=3, λ=0.30 (ρ=0.90)	$L_q = \lambda W_q$ (S-general)	2.7000	2.7118	0.44%
S_1=3, λ=0.30 (ρ=0.90)	$W_q = \rho(S-1)/(2(1-\rho))$	9.0000	9.0416	0.46%
S_1=3, ratio vs. S=2-only form	$\hat{L}_q/[\lambda^2(S-1)/(1-\lambda S)]$	1.5000	1.5066	0.44%
V2: Single-Station Geometric Service ($Geo/Geo/1$, $E[S]$=2) & Factor-2 Law				
$E[S]$=2, λ=0.30 (ρ=0.60)	$L_q = 2\lambda^2/(1-2\lambda)$	0.4500	0.4505	0.11%
$E[S]$=2, λ=0.45 (ρ=0.90)	$L_q = 2\lambda^2/(1-2\lambda)$	4.0500	4.0775	0.68%
$E[S]$=2, worst stable (λ=0.10)	$\max_{\lambda \le 0.45} \lvert L_q - \hat{L}_q \rvert / L_q$	0.0250	0.0248	0.94%
Factor-2 ratio (λ=0.45)	$\hat{L}_q^{\text{Geo}}/\hat{L}_q^{\text{D}}$	2.0000	2.0062	0.31%
Factor-2 sweep ($\lambda \in [0.05, 0.45]$)	Range of $\hat{L}_q^{\text{Geo}}/\hat{L}_q^{\text{D}}$	2.0000	1.984–2.019	$\le 0.95\%$
V3: Full 3-Station Pipeline Accounting (B=1, τ=0, c=1, λ=0.30)				
Flow conservation (Q_1, Q_2, Q_3)	Departures/Arrivals	1.0000	0.9999997	3.3×10^{-7}
Station utilizations ($\hat{\rho}_1, \hat{\rho}_3$)	$\rho_i = \lambda_i E[S_i]$	0.6000	0.5997 / 0.5999	$\le 0.06\%$
Per-node $W = L_q/\lambda$ relation	$\hat{W}_{q,i}/(\hat{L}_{q,i}/\lambda_i)$	1.0000	1.0000000	1.5×10^{-16}
Network total occupancy	$\hat{L}_{\text{tot}}/(\lambda \hat{W}_{\text{e2e,qwait}})$	0.5591	0.5590868	3.3×10^{-7}

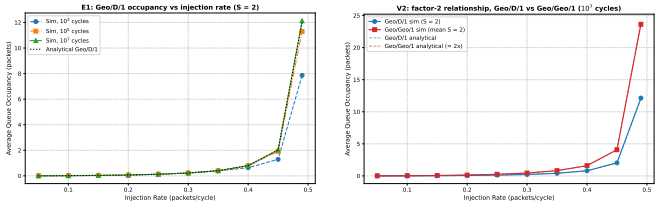


Fig. 2. Layer-1 verification. *Left (V1):* $Geo/D/1$ (S_1=2) queue occupancy across horizons $T \in \{10^3, 10^5, 10^7\}$ cycles, converging onto $L_q = \lambda^2/(1-2\lambda)$ (0.46% error at λ=0.30, 10^7 cycles). *Right (V2):* $Geo/Geo/1$ vs. $Geo/D/1$ at 10^7 cycles, confirming the exact 2× Pollaczek-Khinchine variance penalty.

IV. Layer-1 Analytical Verification

We verify single-station and pipeline behavior across $\lambda \in \{0.05, \dots, 0.45, 0.49\}$ and $T \in \{10^3, 10^5, 10^7\}$ cycles. Additionally, `check_conventions.py` confirms bit-exact integer counter equivalence with the course's `single_queue_simulation_geo_d_1.m` across four (S, λ, T) test cases.

TABLE II
LAYER-2 EMPIRICAL TESLA T4 GPU VALIDATION, UNCAPPED PREDICTOR COMPARISON & MULTI-MODEL BATCH-SCALING SWEEPS

Part A: Split Calibration ($B \in \{1,4,16\}$) & Held-Out Validation ($B \in \{2,8,32\}$), 96-Token Cap						
Batch Size B	**Split Role**	**Colab Mean (s)**	**Kaggle Mean (s)**	**Power-Law Pred. (s)**	**Latency Error**	**Measured / Pred. Req/s (Error)**
1	Calib.	3.868 ± 0.29	4.069	4.030	—	0.259 / 0.248 (—)
4	Calib.	4.382 ± 2.21	3.826	4.036	—	0.913 / 0.991 (—)
16	Calib.	3.879 ± 0.29	3.980	4.042	—	4.125 / 3.959 (—)
2	**Held-Out**	$\mathbf{3.795 \pm 0.28}$	**3.869**	**4.033**	**+6.26%**	**0.527 / 0.496 (-5.89%)**
8	**Held-Out**	$\mathbf{3.800 \pm 0.28}$	**3.865**	**4.039**	**+6.29%**	**2.105 / 1.981 (-5.92%)**
32	**Held-Out**	$\mathbf{4.015 \pm 0.30}$	**4.088**	**4.045**	**+0.74%**	**7.970 / 7.912 (-0.74%)**
Held-Out Mean Absolute Percentage Error (MAPE):					**4.43%**	**Throughput: 4.18%**

Part B: Uncapped 512-Token Structural Sweep (30 Trials, $L_j \in [87, 498]$ Tokens)			
Predictor Model	**Feature Correlation r**	**Held-Out Latency MAPE**	**Throughput MAPE**
Linear-in-B: $\hat{T}(B) = \beta_0 + \beta_1 B$	$r(T, B) = 0.7384$	15.25%	17.60%
Max-Length: $\hat{T}(L_{\max}) = \gamma_0 + \gamma_1 \max_j L_j$	$\mathbf{r(T, L_{max}) = 0.9493}$	**9.14%**	**10.75%**

Part C: Multi-Model Measured T4 Sweeps (Short-QA, $B \in \{1,\ldots,32\}$), Four Model Families					
Model (Trials per B)	**FP16 Wt. / Peak GB (B=32)**	**KV Cache per Token**	**Batch Time (s) B=1 → 32**	**Req/s (Gain) B=1 → 32**	**Scaling Exponent α**
96-token cap (controlled hardware benchmark)					
`Qwen2.5-1.5B` (10)	3.0 / —	28 KB	3.87 → 4.02	0.26 → 7.97 (31×)	0.001
`Gemma-2-2B-it` (6)	5.2 / 5.7	104 KB	4.79 → 5.07	0.21 → 6.31 (30×)	0.012
`Llama-3.2-1B-Inst.` (10)	2.5 / 2.7	32 KB	2.63 → 2.87	0.38 → 11.14 (29×)	0.022
512-token ceiling (uncapped generation lengths)					
`Llama-3.2-1B-Inst.` (5)	2.5 / 3.2	32 KB	9.84 → 22.15	0.10 → 1.45 (14×)	0.20
`Phi-3-mini-4k` (5)	7.6 / 14.5	384 KB	5.58 → 61.29	0.18 → 0.52 (2.9×)	0.67

Deterministic Service ($Geo/D/1$) & S-General Formula. From the waiting-time PGF $G_W(z) = (1-\lambda S)/[1-\lambda(1+z+\cdots+z^{S-1})]$, the exact S-general closed-form pair (`top_script.m`) is:

$$W_q = \frac{\rho(S-1)}{2(1-\rho)}, \qquad L_q = \lambda W_q = \frac{\lambda^2 S(S-1)}{2(1-\lambda S)}. \quad (4)$$

During verification we proved that the Assignment-1 formula $L_q^{\mathrm{A1}} = \lambda^2(S-1)/(1-\lambda S)$ matches our post-dispatch convention *only* at S=2 ($S/2$=1 $\implies$ $L_q = \lambda^2/(1-2\lambda)$), whereas at S=3 the exact occupancy is $S/2 = 1.5\times$ larger. Table I and Fig. 2 (left) show that at 10^7 cycles, our simulator matches Eq. (4) within 0.75% across $\lambda \le 0.45$ for S_1=2 and within 0.46% for S_1=3 ($\lambda \le 0.30$, confirming the $1.5066\times$ ratio). At λ=0.30, L_q relative error converges from 7.56% (10^3 cycles) to 1.83% (10^5) and 0.46% (10^7).

Geometric Service ($Geo/Geo/1$) & V3 Accounting. For geometric service with $E[S]$=2 (Var(S)=2), discrete-time Pollaczek-Khinchine gives $L_q^{\mathrm{Geo}} = 2\lambda^2/(1-2\lambda) = 2L_q^{\mathrm{D}}$. Table I and Fig. 2 (right) confirm agreement within 0.94% across $\lambda \le 0.45$ and an empirical ratio $\hat{L}_q^{\mathrm{Geo}}/\hat{L}_q^{\mathrm{D}} \in [1.9835, 2.0190]$ (0.31% error at λ=0.45). All 87 stable checks in `validation_table.csv` pass with zero failures.

V. LAYER-2 EMPIRICAL GPU ANALYSIS AND SIMULATION RESULTS

To address Phase-1 Feedback #2 (*stronger experiments, diverse models, and heterogeneous queries*), we executed hardware profiling on Nvidia Tesla T4 GPUs alongside multi-parameter simulator campaigns E2 and E3.

A. Empirical GPU Grounding & Split Calibration (Tesla T4, Qwen 2.5-1.5B)

We profiled `Qwen/Qwen2.5-1.5B-Instruct` in FP16 (`sdpa` attention, KV-cache enabled) on Nvidia Tesla T4 GPUs (15.36 GB VRAM, CUDA 12.8, PyTorch 2.11) across Google Colab and Kaggle. We swept static batch sizes $B \in \{1,2,4,8,16,32\}$ with 10 trials per size (60 batch runs, 630 generated sequences) using CUDA-synchronized timers, plus 20 CPU tokenizer trials (0.206 ms mean, SD=0.283 ms, range $[0.068, 1.369]$ ms).

Out-of-Sample Power-Law Calibration. We split the batch sweep into a *calibration set* $B_{\mathrm{cal}} \in \{1,4,16\}$ and an unseen *held-out test set* $B_{\mathrm{test}} \in \{2,8,32\}$. Log-linear OLS regression on B_{cal} yields:

$$T_{\mathrm{batch}}(B) = a \cdot B^{\alpha}, \quad a = 4.0301 \text{ s}, \quad \alpha = 0.001044. \quad (5)$$

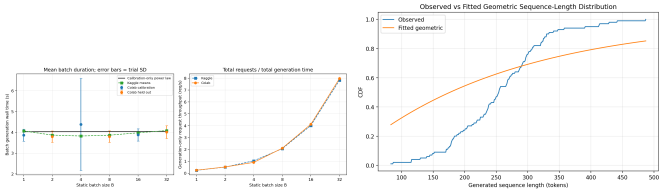


Fig. 3. Layer-2 Tesla T4 GPU measurements (`Qwen2.5-1.5B-Instruct`). *Left:* Batch execution time and throughput across $B \in \{1,\ldots,32\}$ on Colab and Kaggle with power-law calibration on $\{1,4,16\}$ and held-out validation on $\{2,8,32\}$ (4.43% MAPE). *Right:* Uncapped 100-prompt empirical sequence-length CDF vs. fitted geometric CDF ($KS = 0.3965$).

As shown in Table II (Part A) and Fig. 3 (left), predicting on held-out sizes $B \in \{2,8,32\}$ achieves a **Batch Latency MAPE of** 4.43% (individual errors $+6.26\%, +6.29\%, +0.74\%$) and a **Throughput MAPE of** 4.18%. Because $\alpha \approx 0.001$ under the 96-token cap (90–98% of sequences hit 96 tokens), batch latency stays nearly flat ($\approx$ 3.80–4.01 s) from B=1 to 32, scaling throughput $30.8\times$ from 24.59 tok/s (0.259 req/s at B=1) to 763.28 tok/s (7.970 req/s at B=32). Cross-platform Kaggle vs. Colab means agree within 1.70–4.94% across $B \in \{1,2,8,16,32\}$.

B. Query Diversity, Straggler Length & Non-Geometric Distributions

To move beyond fixed 96-token caps and evaluate natural query diversity, we conducted two uncapped diagnostic studies with a 512-token ceiling:

1) *100-Prompt Uncapped Length Diagnostic:* Across 100 diverse prompts, $0/100$ hit the 512-token cap. Generated lengths spanned $[83, 487]$ tokens (median 254.5, mean 255.23, SD=74.57, P90 330.4, P99 442.5 tokens; wall time 3.36–19.80 s, mean 10.30 s). Fig. 3 (right) shows that comparing the empirical CDF against a fitted memoryless geometric distribution yields $\mathbf{KS = 0.3965}$, proving that real LLM outputs exhibit a positive minimum-length floor and heavy concentration around 180–330 tokens rather than memoryless step-1 decay.
2) *Uncapped Straggler Coupling (r=0.9493):* In static batching, a worker holds the batch until the *longest sequence* $L_{\max} = \max_{j \le B} L_j$ completes. Across 30 uncapped batch trials ($B \in \{1,\ldots,32\}$), batch time correlates moderately with B (r=0.7384) only because $E[\max_{j \le B} L_j]$ grows from 232.8 tokens (B=1) to 454.2 tokens (B=32), whereas correlation with $L_{\max}$ is $\mathbf{r = 0.9493}$ ($\approx$ 40.3 ms/token). As shown in Table II (Part B), conditioning on $L_{\max}$ cuts held-out latency MAPE on $B \in \{2,8,32\}$ from $\mathbf{15.25}\%$ (linear-in-B) to $\mathbf{9.14}\%$ and throughput MAPE from $\mathbf{17.60}\%$ to $\mathbf{10.75}\%$.

C. Multi-Model Batch-Scaling Sweeps & Cycle Mapping

Beyond the `Qwen2.5-1.5B-Instruct` calibration ($\approx$ 3.0 GB FP16 weights, HBM weight-read floor $\approx$ 9.4 ms/step at 320 GB/s vs. measured $\approx$ 40.3 ms/step), Table II (Part C) reports measured T4 sweeps on three further model families (`Gemma-2-2B-it`, `Llama-3.2-1B-Instruct`, `Phi-3-mini-4k-instruct`) over the same $B \in \{1,\ldots,32\}$ with 5–10 trials per size. Under the 96-token cap, all three families reproduce the flat service law seen for Qwen: batch time grows by only 4–9% from B=1 to B=32 (α=0.012 for Gemma, 0.022 for Llama), so request

TABLE III
SIMULATION EXPERIMENTS E2 (WORKER SCALING) & E3 (BATCHING TRADE-OFFS, λ=0.30, δ=0.5, 10^7 CYCLES)

Part A: Experiment E2 — Service Variability & Worker Parallelism (B=1, τ=0, 10^6 **Cycles**)							
λ	c1-D W_{soj}	c1-Geo W_{soj}	**Var. Premium**	c2-Geo W_{soj}	$L_{q,1}$ (All)	$L_{q,3}$ (c1-D)	$L_{q,3}$ (c1/c2-Geo)
0.10	4.123	4.303	+4.4%	4.122	0.0123	0.0000	0.0183 / 0.0001
0.20	4.332	4.835	+11.6%	4.340	0.0664	0.0000	0.1004 / 0.0014
0.30	4.754	5.871	+23.5%	4.770	0.2264	0.0000	0.3349 / 0.0044
0.40	6.018	9.067	+50.7%	6.050	0.8074	0.0000	1.2185 / 0.0111
0.45	**8.519**	**15.350**	**+80.2%**	**8.549**	**2.0346**	**0.0000**	**3.0761 / 0.0142**
0.49	28.823	69.151	+139.9%	28.862	12.1750	0.0000	19.7793 / 0.0185

Part B: Experiment E3 — Batching Scheduler Trade-Off Grid (λ=0.30, δ=0.5, c=1, 10^7 **Cycles**)							
B	τ	**Mean Fill** $\bar{b}$	W_{soj} (cyc)	W_{qwait} (cyc)	$L_{q,2}$ (req)	Q_3 **Util.** $\hat{\rho}_3$	**Throughput**
1	**2 or 8**	**1.000**	**4.752**	**0.752**	**0.000**	**0.5999**	**0.3000**
2	2	1.574	6.752	2.023	0.381	0.4906	0.3000
4, 8, 16	2	1.574	6.752	2.023	0.381	0.4906	0.3000
2	8	1.953	7.360	2.384	0.489	0.4536	0.3000
4	8	3.172	10.438	4.875	1.236	0.3224	0.3000
8	**8**	**3.329**	**10.969**	**5.391**	**1.391**	**0.3063**	**0.3000**
16	**8**	**3.329**	**10.969**	**5.391**	**1.391**	**0.3063**	**0.3000**

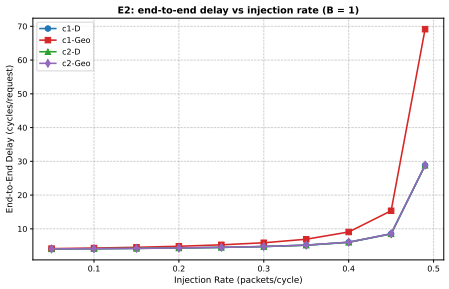


Fig. 4. Experiment E2 (10^6-cycle CRN sweep, B=1): End-to-end sojourn delay (W_{soj}) vs. injection rate λ. Geometric decode variance at c=1 inflates delay by 80.2% at λ=0.45 (15.35 vs. 8.52 cycles), whereas c=2 parallel workers absorb the penalty (8.55 cycles).

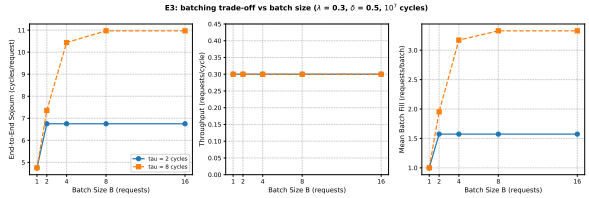


Fig. 5. Experiment E3 (λ=0.30, δ=0.5, 10^7 cycles, CRN-paired): End-to-end sojourn W_{soj}, throughput, and mean batch fill $\bar{b}$ across $B \in \{1, 2, 4, 8, 16\}$ and $\tau \in \{2, 8\}$. At τ=8, fill and delay saturate between B=4 ($\bar{b}$=3.17) and B=8 ($\bar{b}$=3.33).

throughput rises by $\approx$ 29–31×, and measured per-step decode time ($\approx$ 27 ms Llama, $\approx$ 50 ms Gemma) sits 3.1–4.3× above each model's weight-read floor (7.7 and 16.3 ms). Without the cap, scaling is model-dependent: `Llama-3.2-1B` remains strongly sublinear (α=0.20, 14× throughput gain), whereas `Phi-3-mini` (7.6 GB weights, 384 KB KV-cache/token) is close to linear (α=0.67, only 2.9× gain) and peaks at 14.5 GB of the 15.4 GB T4 at B=32 (two larger-batch runs on other workloads ran out of memory). The capped and uncapped rows are separate regimes and are not compared across groups: uncapped Llama batches are dominated by sequences reaching the 512-token ceiling. Finalizing the physical `cycle_ms` mapping in `calibration_parameters.json` is scheduled for Phase 3.

D. Experiment E2: Service Variability & Worker Parallelism

Experiment E2 (`experiment_e2_pipeline.csv`) sets Q_2 to pass-through (B=1, τ=0) and compares deterministic (S_3=2) vs. geometric ($E[S_3]$=2) GPU service across $c \in \{1, 2\}$ under CRN-paired arrivals. Table III (Part A) and Fig. 4 show: **(1) Structural Zero-Queue Property:** In all 22 deterministic runs ($c \in \{1, 2\}$, all λ), $L_{q,3} = 0.0000$ identically: Q_1 (S_1=2) spaces departures by ≥ 2 cycles, so a 2-cycle Q_3 service finishes before the next arrival dispatches. **(2) Variability Premium at c=1:** Geometric Q_3 service builds a large queue ($L_{q,3}$=3.076 at λ=0.45), inflating sojourn by +23.5% at λ=0.30 (4.754 → 5.871 cycles), +80.2% at λ=0.45 (8.519 → 15.350 cycles), and +139.9% at λ=0.49 (28.823 → 69.151 cycles). **(3) Parallel Absorption (c=2):** Adding a second worker (c2-Geo) cuts $L_{q,3}$ at λ=0.45 to 0.0142, reducing sojourn from 15.350 to 8.549 cycles (within 0.030 cycles of deterministic 8.519).

E. Experiment E3: Static Batching Scheduler Trade-Offs

Experiment E3 (`experiment_e3_batching.csv`) sweeps $B \in \{1, 2, 4, 8, 16\} \times \tau \in \{2, 8\}$ at λ=0.30 with sublinear batch service $S_3(b) = \lceil 2 + 0.5(b-1) \rceil$ (δ=0.5, c=1) over 10^6 and 10^7 cycles (agreeing within $\leq$ 0.0038 cycles / 0.29%). Table III (Part B) and Fig. 5 demonstrate: **(1) Delay vs. Utilization Pareto Trade-Off:** Unbatched serving (B=1) minimizes end-to-end sojourn (W_{soj}=4.752 cycles, W_q=0.752 cycles, $L_{q,2}$=0) but requires high GPU worker utilization ($\hat{\rho}_3$=0.5999). Batching at τ=8, $B \geq 8$ achieves mean fill $\bar{b}$=3.329 (3.400 cycles/batch), slashing GPU utilization by **48.9%** ($\hat{\rho}_3$: 0.5999 → 0.3063) while raising Q_2 wait (W_{soj}=10.969, W_q=5.391 cycles). **(2) Short-Timer Invariance (τ=2):** At τ=2, the timer fires before B requests accumulate behind Q_1, making $B \in \{2, 4, 8, 16\}$ identical ($\bar{b}$=1.574, W_{soj}=6.752, $\hat{\rho}_3$=0.4906). **(3) Saturation Knee (τ=8):** Fill rises from 1.953 (B=2) to 3.172 (B=4) and saturates at 3.329 for $B \in \{8, 16\}$ (identical rows), while throughput stays invariant at 0.3000 req/cycle across all 20 runs.

VI. LIMITATIONS, CONCLUSION, AND PHASE-3 PLAN

Limitations & Phase-3 Plan. Our simulator models a feedforward chain with Bernoulli arrivals, static batching, and non-preemptive GPU service. In Phase 3, we will: (1) extend sweeps in Table II to summarization and code workloads, calibrating model-specific slopes δ; (2) finalize the physical `cycle_ms` mapping; and (3) incorporate network bounds (V4) and empirical $L_{\max}$ sampling at Q_3.

Conclusion & Code Availability. This Phase-2 report resolves both Phase-1 feedback items via an explicit systems-to-queueing latency decomposition verified to $< 0.95\%$ over 10^7 cycles, multi-model Tesla T4 GPU sweeps across four model families (`Qwen2.5-1.5B` at 4.43% held-out MAPE, `Gemma-2-2B`, `Llama-3.2-1B`, `Phi-3-mini-4k`), 100 uncapped diagnostics (KS=0.3965, r=0.9493, 9.14% MAPE), and E2/E3 trade-off analyses. Full code and measurement traces are publicly available at https://github.com/vaibhav3000/CSA-Modelling-Simulation-Project and https://github.com/h2o-Noob/MS-Project.

REFERENCES

[1] W. Kwon *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. ACM SOSP*, 2023, pp. 611–626.
[2] G.-I. Yu *et al.*, "Orca: A distributed serving system for transformer-based generative models," in *Proc. USENIX OSDI*, 2022, pp. 521–538.
[3] Y. Sheng *et al.*, "FlexGen: High-throughput generative inference of large language models with a single GPU," in *Proc. ICML*, 2023, pp. 31094–31116.
[4] A. Agrawal *et al.*, "Vidur: A large-scale simulation framework for LLM inference," in *Proc. MLSys*, 2024.
[5] M. Haviv, *Queues: A Course in Queueing Theory*. Springer, 2013, vol. 191.
[6] L. Zheng *et al.*, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in NeurIPS*, vol. 36, 2023, pp. 46595–46623.